Device Health Monitoring Automation with Claude: Research and Content Plan

Network and Infrastructure Automation · 3 days (8 hours per day) · Intermediate · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.

- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References

1. Where this course came from

Client input.

> Client use case 6, 'Device Health Monitoring Automation: Monitoring health status of devices by checking VIP status, uptime status, HA status, sync status if in HA, version, last change performed on devices, updating or creating SNOW ticket accordingly', with the client's aim 'to do automation using Claude for all these content'. Use case 1 names the load balancer fleet: F5, NetScaler, Radware and Anaya LB.

Load balancer and fabric health is still checked by hand in many NOCs: someone logs in to each pair, looks at VIPs, failover and sync, and raises a ticket if something looks wrong. The signals are well defined on every platform, from F5 failover and sync states to NetScaler HA node statistics and APIC fabric health [11][14][2][26], but reading them consistently across vendors, every few minutes, is the part people skip.

The hard failures are often quiet. An HA pair with mismatched versions keeps passing traffic while config sync is disabled [21][14]. A second ticket gets raised for a fault that already has one. Automation fixes collection and deduplication with plain code; Claude adds value by reading the evidence, linking it to the last change and writing a cited verdict and ticket text a NOC engineer can act on [32].

The client asked for exactly this: the six signals, and updating or creating the ServiceNow ticket accordingly. The course builds that platform with read-only device access, approvals for consequential ticket actions, and an audit trail.

2. How the outline was shaped

Decision	Rationale
Use the client's six signals as the data model.	The client input lists VIP status, uptime, HA, sync if in HA, version and last change; making those the schema keeps every module tied to the request.
Collectors are deterministic code; Claude only analyses.	Up or down must not depend on a model; Claude receives readings and returns a schema-validated verdict [32].
Unknown is never healthy.	Partial data is common with mixed vendors and SNMP-only devices; marking it unknown prevents false calm.
Update or create, never duplicate.	The client said 'updating or creating SNOW ticket accordingly'; a correlation key and query-before-create through the Table API make that testable [5].
Cover Anaya LB through a generic adapter.	We found no public API documentation for a product named Anaya LB, so the course teaches an adapter interface any CLI, SNMP or REST device can implement.
Run on GitHub Actions with a self-hosted runner.	Gives scheduling, reviewed changes and secrets with tools the audience already uses, while devices stay on private networks [7][37].

3. Audience and prerequisites

Role	What they need	Starting point
Load balancer engineer	Automated, trustworthy health checks across vendors	Strong F5 or NetScaler CLI, basic Python
NOC lead	Fewer duplicate tickets and clear verdicts	Knows ServiceNow and escalation, light coding
Data center network engineer (ACI)	Fabric health in the same pipeline	ACI operations, some REST
ITSM or automation engineer	Reliable Table API and Flow Designer integration	ServiceNow admin skills, Python

Prerequisites from the outline

- Working knowledge of load balancers (VIPs, pools, HA pairs, config sync)
- Basic REST API and JSON concepts
- Basic Python and Git
- Familiarity with ServiceNow incidents and change requests is helpful, not required

Checks before enrolment

- Can you explain active/standby and config sync on an F5 or NetScaler pair?
- Can you call a REST API with curl and read the JSON?
- Can you write and run a small Python script and use Git?
- Do you know what an incident and a change request are in ServiceNow?

Course outcomes

- Define a platform-neutral health model covering VIP status, uptime, HA state, sync state, version and last change for every device class
- Collect health signals from F5 iControl REST and tmsh, Citrix NetScaler NITRO, Radware Alteon via Cyber Controller, other load balancers (Anaya LB) and Cisco APIC REST
- Use Claude with tool use and structured JSON outputs to turn raw evidence into a cited, severity-ranked health verdict
- Create or update ServiceNow incidents through the Table API and Flow Designer with deduplication and correlation IDs
- Run the whole workflow on a schedule under approvals, guardrails and a complete audit trail

4. Lab environment

A lab fleet per cohort with fictional hostnames: an F5 BIG-IP VE pair (lb-f5-01/02), a NetScaler VPX pair (lb-ns-01/02), a Radware Alteon VA (lb-alt-01), a generic Linux-based load balancer standing in for Anaya LB, and an APIC (lab APIC or DevNet simulator), plus a ServiceNow PDI, GitHub organization with a self-hosted runner, and Prometheus and Grafana.

Components

Gennoor Tech TRAINING INNOVATE BUILD
For more info - https://gennoor.com

Component	Version or edition	Purpose	How to obtain	Notes
F5 BIG-IP VE	17.x (two instances)	F5 HA pair	30-day trial from MyF5 Trials, or a lab license from a reseller	Trial limited to 1 Mbps; confirm two trial instances are allowed or use lab licenses [45].
NetScaler VPX Express	14.1 (and one 13.1 image for the mismatch lab)	NetScaler HA pair	Free VPX Express, no license file	20 Mbps, 250 SSL sessions; confirm HA pair support on Express before the cohort [41].
Radware Alteon VA	AlteonOS 35.0 (AWS listing) or the client's version	Alteon collector	AWS Marketplace 30-day free trial, then hourly	Stop instances after labs to control cost [24].
Generic load balancer (Anaya LB stand-in)	HAProxy or Keepalived on Linux	Generic adapter practice	Open source packages	Replace with the client's Anaya LB if access is available.
Cisco APIC	Lab APIC or DevNet ACI Simulator	ACI health collector	DevNet sandbox, free	Always-On is shared; reservable sandbox up to 5 days [42].
ServiceNow PDI	Current release	Incidents, changes and flows	ServiceNow Developer Site	Request early [43].
Claude Code, Agent SDK and MCP Python SDK	Current releases; Python 3.11+	Agent and tools	Native installer, pip install claude-agent-sdk, pip install "mcp[cli]"	Requires a Claude account with Claude Code access; MCP server built with the Python SDK [31].
GitHub with Actions and self-hosted runner	GitHub.com	Schedule and reviews	Free organization	Runner needs outbound 443 only [37].
Prometheus and Grafana	Current OSS releases	Dashboards	Docker images	Use textfile collector on the runner host [39].

Fixtures and sample data

Fixture	Contents	Used in
Recorded API responses	F5, NITRO, Alteon SNMP walks, APIC classes, including errors	Collecting health from F5 and NetScaler; Collecting health from Radware, other load balancers (Anaya LB) and Cisco ACI
Synthetic evidence bundles	Twenty bundles with expected severity	Claude as the triage analyst: structured health verdicts
ServiceNow seed data	CIs, an open incident, a closed change, a scheduled change	Creating or updating ServiceNow tickets; Scheduling and running the workflow with GitHub Actions
Fault scripts	Break sync, downgrade one NetScaler node, disable a VIP, revoke a credential	Capstone: end-to-end device health monitoring for a mixed fleet

Lab safety rules

- Nothing touches production.
- Device accounts used by collectors are read-only.
- Fault scripts run only on lab devices by the instructor or under supervision.
- Secrets only in environment variables and GitHub secrets.
- Stop cloud instances after each day.

5. Module-by-module content plan

Day 1

Module 1: What device health means across the fleet

Goal. Learners define six platform-neutral health signals and record what healthy looks like on every platform in the lab fleet.

Outline topics.

- The six health signals: VIP status, uptime, HA status, sync status when in HA, software version and last change performed
- How each signal looks on F5 BIG-IP, Citrix NetScaler, Radware Alteon, other load balancers (Anaya LB) and Cisco ACI
- Why manual morning checks fail: missed sync drift, stale tickets, duplicate incidents
- The pattern for the whole course: Claude plans and explains, deterministic code executes, a person approves, everything is logged

Learning objectives.

Gennoor Tech TRAIN. INNOVATE. EMPOWER. For more info - https://gennoor.com

- Learners can define VIP status, uptime, HA status, sync status, software version and last change as signals with allowed values.
- Learners can find each signal on F5 BIG-IP, NetScaler, Radware Alteon, a generic load balancer and Cisco ACI [3][1][23][26].
- Learners can explain why manual morning checks miss sync drift and create duplicate tickets.
- Learners can state the course pattern: Claude explains, code collects and decides, a person approves, everything is logged.

Six signals and their allowed values

- VIP status: up, down, disabled, unknown, per virtual server, plus a count of up versus total.
- Uptime: seconds since start, so a recent reboot stands out.
- HA status: role (active/standby or primary/secondary) and health of the pair.
- Sync status: only when the device is in HA; values normalised to in_sync, pending, failed, unknown.
- Version: the running software version and whether both HA peers match; F5 requires all device group members on the same version and NetScaler disables HA sync on mismatched versions [14][21].
- Last change: time, actor and source (device log, Git diff or change record).

How each platform shows the signals

- F5: virtual stats status.availabilityState, failoverState in cm/device, cm/sync-status, version and build in cm/device [12][11][10].
- F5 sync statuses: In Sync, Changes Pending, Awaiting Initial Sync, Not All Devices Synced, Disconnected [14].
- NetScaler: lbvserver state UP(7)/DOWN(1)/OFS(4), hanode master state and haerrsyncfailure, nsversion, system timesincestart [1][2][17][20].
- Radware Alteon: SNMP MIBs for Layer 4 virtual services and HA state, plus the Alteon REST API documented in the Alteon REST API User Guide [23][9].
- Cisco ACI: fabricHealthTotal, topSystem and fabricNode class queries, firmware from firmwareRunning [26][44].

Why manual checks fail

- Nobody reads sync status on the standby unit, so drift sits for days.
- A ticket from yesterday is still open and a second one is raised for the same fault.
- Version mismatch after half an upgrade looks healthy at a glance but breaks HA sync [21].
- Checks are not recorded, so nobody can prove what was looked at.

The working pattern

- Collectors are deterministic Python; Claude never decides whether a reading is up or down.
- Claude turns an evidence bundle into a cited verdict and a draft ticket text.
- Consequential actions (P1/P2, reopen, paging) wait for a person.

- Every run is written to an audit record; MCP itself expects tool use to be logged [30].

Instructor demo.

1. On lb-f5-01 run tmsh show sys failover and tmsh show cm sync-status, then the REST equivalents [13][10].
2. On lb-ns-01 call /nitro/v1/stat/lbvserver and /nitro/v1/stat/hanode and point at state and haerrsyncfailure [1][2].
3. On the APIC open fabricHealthTotal and fabricNode class queries [26].
4. Read an Alteon HA state over SNMP with snmpget against the lab VA [23].
5. Fill the first row of the health signal matrix live.

Hands-on lab. Walk the lab fleet by hand with tmsh, NITRO and the APIC GUI and record what a healthy baseline looks like for each platform.

1. Log in to lb-f5-01 and lb-f5-02 and record failover state, sync status, version and uptime [13] [14].
2. Call the F5 virtual stats endpoint for two VIPs and record availabilityState [12].
3. On lb-ns-01 and lb-ns-02 record lbvserver states, show ha node output, nsversion and timesincestart [1][19][17][20].
4. On lb-alt-01 record virtual service state, HA state and uptime over SNMP [23].
5. On the APIC record fabric health, node count and versions [26][44].
6. For the generic device record what is available by CLI or SNMP only.
7. Fill the health signal matrix with source, path and healthy value for each signal.
8. Mark any signal you could not read as unknown with the reason.

Lab success criteria.

- The matrix has all six signals for all five platforms with the exact command or endpoint.
- Every healthy value is written as the raw platform value and the normalised value.
- Unknowns are explicit, never blank.
- Sync status is marked not applicable for standalone devices.

Deliverable. Health signal matrix per platform.

Common pitfalls.

- Reading sync on only one member of a pair.
- Treating NetScaler OFS or F5 disabled as down.
- Assuming the ACI health score maps directly to a device verdict.
- Leaving a signal blank instead of unknown.

Knowledge check.

1. Which F5 sync status means at least one member lacks the latest configuration? Answer: Not All Devices Synced [14].

2. What happens to NetScaler HA config sync when peers run different versions? Answer: It is disabled, along with command propagation and connection mirroring [21].
3. Which NetScaler stat counts config sync failures since the last transition? Answer: haerrsyncfailure in /nitro/v1/stat/hanode [2].
4. Why is sync status conditional? Answer: It only applies when the device is in an HA pair; standalone devices report not applicable.

Slides and assets to prepare.

- Health signal matrix template (platform x signal).
- Lab fleet diagram with fictional hostnames.
- Raw-to-normalised value mapping table.
- Command and endpoint cheat sheet per platform.

Sources for this module. 1, 2, 3, 9, 10, 11, 12, 13, 14, 17, 19, 20, 21, 23, 26, 30, 44

Module 2: Setting up Claude Code, the Agent SDK and MCP for this workflow

Goal. Learners scaffold the project, run a read-only device MCP server and have the agent read a device version through a tool call, with credentials kept out of the model context.

Outline topics.

- Installing Claude Code, project CLAUDE.md conventions and the Agent SDK in Python
- Building a read-only MCP server that exposes get_vip_status, get_ha_state and get_version tools over the lab devices
- Tool use and structured JSON outputs: defining a HealthReading schema with Pydantic
- Secrets handling for device and ServiceNow credentials outside the model context

Learning objectives.

- Learners can create a project with CLAUDE.md conventions and confirm it loads [28].
- Learners can build a read-only MCP server with get_vip_status, get_ha_state and get_version [30][31].
- Learners can define a HealthReading Pydantic schema used by both tools and agent.
- Learners can keep device and ServiceNow credentials outside the model context [29].

Project and CLAUDE.md

- Install Claude Code, run claude doctor, and run /init in the repository [28].
- CLAUDE.md conventions: collectors are pure functions, no write calls to devices, unknown is never healthy, every verdict cites readings.
- Import the severity rules file with @rules/severity.md so the agent sees the same rules the code enforces [28].
- CLAUDE.md guides; permissions and hooks enforce [28][33].

Read-only MCP server

- pip install "mcp[cli]" and register tools with @mcp.tool() [31].
- Tools take a device name from inventory, never a URL or credentials.
- Each tool returns a HealthReading as structuredContent with an outputSchema [30].
- API failures return isError: true with a message, so the agent can mark the signal unknown [30].
- Rate limit and validate inputs in the server, as the spec requires [30].

The HealthReading schema

- Fields: device, platform, signal, raw_value, normalised_value, source_path, collected_at, error.
- Enums for normalised values so 'mostly up' cannot appear.
- The same model is used by collectors, MCP tools and tests.
- JSON Schema from model_json_schema() is reused for the agent's output schema [32].

Secrets outside the model

- Credentials live in environment variables loaded by the MCP server process, referenced in .mcp.json with ${VAR} [29].
- F5 uses short-lived tokens from /mgmt/shared/authn/login in the X-F5-Auth-Token header [16].
- NetScaler uses X-NITRO-USER/X-NITRO-PASS or a session cookie, set inside the server only [2].
- Deny the agent Bash and file reads of .env with permission rules [34].

Instructor demo.

1. Scaffold with Claude Code and show CLAUDE.md and the rules import [28].
2. Run the MCP server in the Inspector and call get_version for lb-f5-01 [31].
3. Register with claude mcp add at project scope and show .mcp.json with ${F5_USER} [29].
4. Ask the agent 'What version is lb-f5-01 running?' and show the tool call and answer.
5. Ask it to print the F5 password and show it has no way to reach it.

Hands-on lab. Scaffold the project with Claude Code, run the MCP server locally and have the agent read the version of lb-f5-01 through a tool call.

1. Create the repository and CLAUDE.md with the course conventions [28].
2. Write the HealthReading model.
3. Implement get_version for F5 using cm/device version and build [11].
4. Implement get_ha_state and get_vip_status for F5 [11][12].
5. Test in the MCP Inspector [31].
6. Register the server with environment-variable credentials [29].
7. Add deny rules for Bash and .env reads [34].
8. Ask the agent for lb-f5-01's version and save the transcript.

Gen Noor Tech
For more info - https://genoor.com

Lab success criteria.

- The agent answers with a tool call and cites the source path.
- No credential appears in the repo, transcript or model context.
- A broken credential produces an isError tool result, not a crash.
- All tools return schema-valid HealthReading objects.

Deliverable. Working agent project with a read-only device MCP server.

Common pitfalls.

- Passing credentials as tool arguments.
- Using basic auth for every F5 call instead of a token [16].
- Free-text tool results the agent has to parse.
- Forgetting project-scoped servers need approval on first use [29].

Knowledge check.

1. How do you keep the F5 password out of .mcp.json? Answer: Reference it as ${F5_PASS} and set the variable in the environment [29].
2. What header carries an F5 auth token? Answer: X-F5-Auth-Token [16].
3. What does a tool return when the device is unreachable? Answer: A result with isError: true and a message; the reading becomes unknown [30].
4. Why is the HealthReading model shared? Answer: So collectors, tools, agent output and tests all validate against one schema.

Slides and assets to prepare.

- Starter repository with CLAUDE.md and rules file.
- HealthReading model and JSON Schema.
- .mcp.json and settings.json examples.
- Diagram of where secrets live.

Sources for this module. 2, 11, 12, 16, 28, 29, 30, 31, 32, 33, 34

Module 3: Collecting health from F5 and NetScaler

Goal. Learners write F5 and NetScaler collectors that normalise every reading into HealthReading with retries, timeouts and recorded fixtures.

Outline topics.

- F5 iControl REST: /mgmt/tm/ltm/virtual/stats for VIP availability, /mgmt/tm/cm/device and /mgmt/tm/cm/sync-status for HA and sync, /mgmt/tm/sys/version
- tmsh equivalents (show sys failover, show cm sync-status, show sys version) for devices without REST access

- NetScaler NITRO: /nitro/v1/stat/lbvserver, /nitro/v1/config/hanode, /nitro/v1/config/nsversion and system uptime
- Normalising every response into the shared HealthReading schema with retries and timeouts

Learning objectives.

- Learners can collect VIP, HA, sync, version and uptime from F5 through iControl REST with tmsh fallbacks [10][11][12][13].
- Learners can collect the same signals from NetScaler through NITRO [1][2][17][18][20].
- Learners can normalise platform values and handle partial failures.
- Learners can test collectors with pytest fixtures.

F5 iControl REST

- Authenticate with POST /mgmt/shared/authn/login and reuse the token for 20 minutes [16].
- VIPs: GET /mgmt/tm/ltm/virtual/<vs>/stats or the collection stats; read status.availabilityState and status.enabledState [12].
- HA: /mgmt/tm/cm/device gives failoverState, version and build per member [11].
- Sync: /mgmt/tm/cm/sync-status (read-only) [10]; map In Sync, Changes Pending, Not All Devices Synced, Disconnected [14].
- Pool member state and monitor status explain why a VIP is down [4].
- Anything from the command line is available over REST, so fall back to tmsh only when REST is blocked [3].

tmsh fallbacks

- tmsh show sys failover for state and time in state [13].
- tmsh show cm sync-status for sync [10][14].
- tmsh show sys version for version, tmsh show ltm virtual for VIPs [12].
- Run over SSH with a read-only role and parse with tested regexes.

NetScaler NITRO

- VIPs: GET /nitro/v1/stat/lbvserver with state UP(7), DOWN(1), OFS(4) and vslbhealth [1].
- HA: GET /nitro/v1/stat/hanode for hacurmasterstate, hacurstate and haerrsyncfailure [2].
- Config view: /nitro/v1/config/hanode gives state, hasync setting and hasyncfailurereason [18].
- Version from /nitro/v1/config/nsversion; uptime from timesincestart in /nitro/v1/stat/system [17] [20].
- CLI fallback: show ha node shows haSync (Sync State) and haSyncFailureReason [19].

Normalising and resilience

- Map raw values to enums in one table per platform, unit-tested.
- Timeout per call and two retries with backoff for reads.

- A failed call yields unknown with the error, never a guess.
- Record fixtures from the lab for every endpoint, including an error response.

Instructor demo.

1. curl the F5 token endpoint and the virtual stats endpoint [16][12].
2. curl NITRO stat/hanode on both lab NetScalers and compare [2].
3. Ask Claude Code to write the F5 collector and review the diff.
4. Run pytest against fixtures and show an unknown result from a 401 fixture.
5. Run both collectors against the lab HA pairs.

Hands-on lab. Write the F5 and NetScaler collectors with Claude Code, inspect every diff, and run them against the lab HA pairs.

1. Record fixtures from lb-f5-01/02 and lb-ns-01/02.
2. Write the F5 auth and session helper [16].
3. Write F5 VIP, HA, sync and version functions [10][11][12].
4. Write NetScaler VIP, HA, version and uptime functions [1][2][17][20].
5. Write mapping tables and unit tests.
6. Add timeouts and retries.
7. Run against the lab and compare with the morning's manual matrix.
8. Commit with fixtures.

Lab success criteria.

- Both pairs produce six signals each, matching the manual baseline.
- A 401 or timeout fixture yields unknown with the error recorded.
- Tests run offline.
- Every diff was reviewed and committed by a learner.

Deliverable. F5 and NetScaler collectors with pytest fixtures.

Common pitfalls.

- Using vslbhealth alone; a vserver can be UP with some services down [1].
- Ignoring hacurstatus NO, which makes other HA stats invalid [2].
- Reading hasync as sync result; it is the enable setting, while failures show in hasyncfailurereason and haerrsyncfailure [18][2].
- Token expiry mid-run [16].

Knowledge check.

1. Which NetScaler lbvserver state value means out of service? Answer: OFS(4) [1].
2. Where do you read F5 failover state over REST? Answer: failoverState in /mgmt/tm/cm/device [11].

Gennoor Tech
For more info - https://gennoor.com

3. What is the NetScaler hasync field? Answer: An ENABLED/DISABLED setting for HA sync, not the sync result [18].
4. What does the collector write when NITRO times out? Answer: An unknown reading with the error and source path.

Slides and assets to prepare.

- F5 and NITRO endpoint map.
- Raw-to-normalised tables.
- Fixture pack for both pairs.
- Collector code skeletons.

Sources for this module. 1, 2, 3, 4, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20

Module 4: Collecting health from Radware, other load balancers (Anaya LB) and Cisco ACI

Goal. Learners complete the collector library with Alteon, a generic adapter and APIC, and prove that missing data surfaces as unknown.

Outline topics.

- Radware Alteon health through Cyber Controller REST and SNMP: virtual server state, VRRP peer status, sync status, image version
- A generic adapter interface for other load balancers (Anaya LB) and devices that only offer CLI or SNMP
- Cisco APIC REST API: topSystem and fabricNode for uptime and role, firmwareRunning for version, fabricHealthTotal for fabric health
- Handling partial data: marking a signal as unknown rather than guessing

Learning objectives.

- Learners can read Alteon virtual service, HA and uptime signals over SNMP and identify the REST references for their version [23][9].
- Learners can implement a generic adapter interface for devices with only CLI or SNMP.
- Learners can collect APIC uptime, role, version and fabric health [26][27][44].
- Learners can show that partial data becomes unknown, never healthy.

Radware Alteon

- SNMP v2c/v3 with ALTEON-CHEETAH-LAYER4-MIB for virtual services and real server status, ALTEON-CHEETAH-SWITCH-MIB and HA group state [23].
- sysUpTime for uptime [23].
- The Alteon REST API is documented in Radware's Alteon REST API User Guide; confirm paths for the lab's AlteonOS version before coding [9].

- Cyber Controller is the multi-device management option; where the client uses it, query it instead of each device, after confirming its API in Radware's documentation.
- Lab: Alteon VA from AWS Marketplace on a 30-day trial [24].

Generic adapter (other load balancers, Anaya LB)

- Interface: get_vips(), get_ha(), get_sync(), get_version(), get_uptime(), each returning HealthReading.
- Implementations: SNMP, SSH CLI with tested parsers, or vendor REST.
- Capability flags so a device that cannot report sync says not_supported.
- Contract tests run every adapter against the same expectations.

Cisco APIC

- aaaLogin and aaaRefresh as in the REST guide [27].
- fabricNode for role and state, topSystem for uptime, fabricHealthTotal for fabric health [26].
- firmwareRunning and firmwareCtrlrRunning for versions [44].
- Faults from faultInst for context in verdicts [26].
- Practice on the DevNet ACI simulator when the lab APIC is busy [42].

Partial data

- Every signal defaults to unknown until a reading proves otherwise.
- A device with three of six signals is reported as partial, not healthy.
- Unknown counts appear in run summaries and dashboards.
- Tests feed fixtures with missing fields and assert unknown.

Instructor demo.

1. snmpwalk the Alteon VA HA and virtual service tables [23].
2. Show the generic adapter interface and one SSH implementation.
3. Query fabricHealthTotal and firmwareRunning on the APIC [26][44].
4. Remove a field from a fixture and show the reading becomes unknown.

Hands-on lab. Add the Alteon, generic and APIC collectors and prove a missing signal surfaces as unknown, not healthy.

1. Configure SNMPv3 on the lab Alteon VA and record a walk [23].
2. Write the Alteon collector against the walk fixture.
3. Write the generic adapter interface and one implementation.
4. Write the APIC collector with aaaLogin and class queries [26][27].
5. Add contract tests for all adapters.
6. Add a fixture with no sync data for Alteon.
7. Run the whole library against the lab fleet.
8. Commit.

Gennoor Tech
INNOVATE BUILD
For more info - https://gennoor.com

Lab success criteria.

- Five platforms produce readings in one schema.
- Alteon sync shows unknown when no data, not in_sync.
- APIC readings include fabric health and versions.
- Contract tests pass for every adapter.

Deliverable. Complete multi-platform collector library.

Common pitfalls.

- Hard-coding Alteon OIDs without checking the MIB for the version.
- Using SNMPv2c community strings in the repository.
- Treating ACI fabric health 97 as a device verdict.
- Letting a not-supported signal count as healthy.

Knowledge check.

1. Which MIB holds Alteon Layer 4 virtual service data? Answer: ALTEON-CHEETAH-LAYER4-MIB [23].
2. Which APIC class gives fabric health? Answer: fabricHealthTotal at /api/node/class/fabricHealthTotal.json [26].
3. What does the generic adapter return for a signal the device cannot report? Answer: not_supported, which the verdict treats as unknown, never healthy.
4. Where do you confirm Alteon REST paths? Answer: In Radware's Alteon REST API User Guide for the version in use [9].

Slides and assets to prepare.

- Alteon MIB map.
- Generic adapter interface spec.
- APIC query card.
- Partial-data test fixtures.

Sources for this module. 9, 23, 24, 26, 27, 42, 44

Day 2

Module 5: Last change performed: correlating config history and change records

Goal. Learners build a per-device timeline that links a health change to the last change performed, from device logs, Git snapshots and ServiceNow change records.

Outline topics.

- F5 audit log via tmsh show sys log audit (or /var/log/audit) and sync events in /var/log/ltm, NetScaler ns.log and config save timestamps, APIC audit log via the aaaModLR class

Gennoor Tech TRAIN · INNOVATE · BUILD · for more info - https://gennoor.com

- Pulling closed and scheduled change requests from the ServiceNow change_request table for each configuration item
- Git-tracked configuration snapshots and diffs as a second source of truth
- Building a timeline that links a health change to the last change performed on the device

Learning objectives.

- Learners can extract config changes from F5 audit logs, NetScaler ns.log and APIC aaaModLR [15][22][25].
- Learners can pull change requests for a configuration item from ServiceNow [35][5].
- Learners can store and diff configuration snapshots in Git.
- Learners can merge sources into a timeline and link it to a health change.

Device-side change history

- F5: configuration audit messages go to /var/log/audit, logins to /var/log/secure; check that MCP audit logging is not at the default Disable [15].
- F5 sync problems are investigated in /var/log/ltm [14].
- NetScaler: ns.log in /var/log holds local logs; forward to syslog for longer history [22].
- APIC: aaaModLR tracks who changed what and when; a class query returns the whole fabric; no export policy exists by default [25][26].

ServiceNow change records

- Query change_request with sysparm_query on cmdb_ci and a date range, ordered by end date [5].
- Or use GET /sn_chg_rest/change with sysparm_query [35].
- Use sysparm_fields to fetch only number, state, start_date, end_date, short_description [5].
- Keep both closed and scheduled changes: closed ones explain, scheduled ones suppress.

Git snapshots

- Nightly read-only export of running config into a Git repository per device.
- Diffs between snapshots give exact changed lines.
- Commit metadata records the collection time, not the change time.
- Mask secrets before commit.

The timeline

- Events from all sources normalised to time, device, actor, source, summary.
- Link rule: the last change before the first unhealthy reading, within a window.
- The NetScaler example: a change upgraded one node only; mismatched versions disable HA sync [21].
- Note the right order: NetScaler upgrades the secondary first, so a primary-only upgrade is a process deviation worth flagging [21].

Gemnoor Tech INNOVATE BUILD
For more info - https://gemnoor.com

Instructor demo.

1. Show a config change in F5 /var/log/audit after a tmsh modify [15].
2. Query aaaModLR on the APIC for today [25][26].
3. Query ServiceNow for changes on lb-ns-02 [5].
4. Show the merged timeline for lb-ns-02.

Hands-on lab. Make a controlled change on lb-ns-02, then watch the pipeline find it in the device log, the Git snapshot and the ServiceNow change record.

1. Raise a lab change in the PDI for lb-ns-02 [43].
2. Make the approved config change on lb-ns-02.
3. Pull the change lines from ns.log [22].
4. Take a Git snapshot and diff.
5. Query the change record by cmdb_ci [5].
6. Merge into the timeline.
7. Run the collectors and link the reading to the change.
8. Save timeline.json.

Lab success criteria.

- The change appears from all three sources with consistent times.
- The timeline links it to the reading.
- Secrets masked in Git.
- Missing sources reported, not hidden.

Deliverable. Change timeline per device.

Common pitfalls.

- F5 MCP audit left at Disable so GUI changes are missing [15].
- ns.log rotated before the run [22].
- Time zone mismatch between device logs and ServiceNow.
- Committing secrets in config snapshots.

Knowledge check.

1. Where does F5 write configuration audit messages? Answer: /var/log/audit [15].
2. Which APIC class tracks user changes? Answer: aaaModLR [25].
3. Which Table API parameter limits returned fields? Answer: sysparm_fields [5].
4. Why was the primary-only NetScaler upgrade a problem? Answer: Mismatched versions disable HA sync, and the documented order is secondary first [21].

Slides and assets to prepare.

- Timeline schema and example.

Gennoor Tech TRAIN · INNOVATE · BUILD · For more info - https://gennoor.com

- Log location table per platform.
- ServiceNow query examples.
- Git snapshot job.

Sources for this module. 5, 14, 15, 21, 22, 25, 26, 35, 43

Module 6: Claude as the triage analyst: structured health verdicts

Goal. Learners build a triage agent that turns evidence bundles into cited, severity-ranked HealthVerdicts, with severity rules enforced in code.

Outline topics.

- A HealthVerdict schema: severity, affected signals, cited evidence, likely cause, recommended action, confidence
- Prompting with evidence bundles and forbidding conclusions without a cited reading
- Severity rules as code: sync drift in an HA pair is never below major, a single VIP down on a standby is informational
- Testing the analyst with contradictory and missing evidence

Learning objectives.

- Learners can define a HealthVerdict schema and get it as structured output [32].
- Learners can write a prompt that forbids conclusions without cited readings.
- Learners can implement severity rules as code that override the model.
- Learners can test the agent with contradictory and missing evidence.

The verdict schema

- Fields: device, severity (info, minor, major, critical), affected_signals, evidence[] (reading ids), likely_cause, recommended_action, confidence, unknowns[].
- Pass as output_format and read structured_output; treat error_max_structured_output_retries or a missing structured_output as failure [32].
- Draft-07 schema; enums for severity [32].
- Every evidence id must exist in the bundle, checked in code.

Prompting with evidence bundles

- Bundle: readings, timeline, open tickets, maintenance windows.
- Rule: no citation, no conclusion; unknowns listed explicitly.
- Give platform facts the agent may use, for example that version mismatch disables NetScaler HA sync and F5 needs the same version in a device group [21][14].
- Ask for recommended actions as steps a person takes, never commands the agent runs.
- Build it with the Agent SDK in Python, which runs the same agent loop as Claude Code with hooks, permissions and MCP support, so the triage agent can call the read-only device tools when a reading needs refreshing [8].

Gennoor Tech TRAINING ACADEMY BUILD
For more info: https://gennoor.com

Severity rules in code

- Sync failed or pending in an HA pair: at least major.
- A single VIP down on a standby: info.
- Any signal unknown on an active unit: at least minor.
- Code takes max(model severity, rule severity) and records both.

Testing the analyst

- Twenty synthetic bundles with expected severity and required citations.
- Contradictions: REST says in sync, CLI says pending.
- Missing: no sync data on an HA pair.
- Score: rule agreement, citation validity, unknown handling.

Instructor demo.

1. Show the lb-ns-02 bundle and run the agent.
2. Show the verdict citing hanode and nsversion readings [2][17].
3. Feed a bundle where the agent says minor for sync failure and show code raising to major.
4. Run the test suite and show the score.

Hands-on lab. Build the triage agent with the Agent SDK, feed it twenty synthetic evidence bundles and score its verdicts against the rules.

1. Write HealthVerdict model.
2. Write the prompt and bundle builder.
3. Call the agent with output_format [32].
4. Implement citation checks.
5. Implement severity rules.
6. Run twenty bundles.
7. Score and inspect failures.
8. Tighten prompt or rules and re-run.

Lab success criteria.

- All verdicts are schema-valid or recorded as failures.
- Final severity always respects rules.
- No verdict cites a reading that does not exist.
- Missing evidence produces unknowns.

Deliverable. Triage agent with verdict test suite.

Common pitfalls.

- Letting confidence text replace unknowns.
- Not checking citation ids.
- Rules in the prompt only.

Gennoor Tech TRAIN · INNOVATE · BUILD · For more info - https://gennoor.com

- Ignoring a success result with no structured_output [32].

Knowledge check.

1. Who sets final severity? Answer: Code: the higher of the rule severity and the model severity.
2. What severity is sync drift in an HA pair? Answer: Never below major.
3. What do you do with a success result that has no structured_output? Answer: Treat it as a failure [32].
4. Why must evidence ids be checked? Answer: To stop conclusions resting on invented readings.

Slides and assets to prepare.

- HealthVerdict schema.
- Prompt template.
- Twenty synthetic bundles with expected results.
- Scoring script.

Sources for this module. 2, 8, 14, 17, 21, 32

Module 7: Creating or updating ServiceNow tickets

Goal. Learners build a ticket adapter that updates, reopens or creates incidents through the Table API without ever creating duplicates.

Outline topics.

- ServiceNow Table API: incident create, query by correlation_id and cmdb_ci, update work notes and state
- Decision logic: update the open ticket, reopen a resolved one, or create new, and never create duplicates
- Flow Designer flows that assign, notify and set priority from the verdict severity
- Idempotent runs and dry-run mode that prints the ticket payload without sending it

Learning objectives.

- Learners can create, query and update incidents through the Table API [5][6].
- Learners can implement the update, reopen or create decision with a correlation key.
- Learners can use Flow Designer for assignment, notification and priority [36].
- Learners can run idempotently and in dry-run mode.

Table API basics

- POST /api/now/table/incident inserts one record and returns 201 with a Location header [5].
- The REST API Explorer walkthrough needs admin, web_service_admin or rest_api_explorer [6].
- GET with sysparm_query and sysparm_fields to find existing tickets [5].

- PATCH /api/now/table/incident/{sys_id} to add work_notes or change state [5].

Correlation and decisions

- Correlation key: device:signal, for example lb-ns-02:ha-sync, stored on the incident (correlation_id is the usual field; confirm it in the PDI dictionary) and also in the adapter's own state.
- Open ticket found: update work notes.
- Resolved within reopen window: reopen, which needs approval.
- None: create.
- Always query before create, and lock per key during a run.

Flow Designer

- Trigger on incident created or updated with the correlation prefix.
- Assign by cmdb_ci support group, notify, and set priority from impact and urgency mapped from severity.
- Use Ask for Approval with a Due Date where the process needs a human step [36].
- Keep business logic in one place; the adapter does not duplicate flow logic.

Idempotence and dry run

- Same input twice gives one ticket and one extra work note.
- Dry run prints the payload and decision without calling ServiceNow.
- Every decision is written to the audit record.
- Back off on 429 and 5xx responses.

Instructor demo.

1. Create an incident with curl and show the Location header [5].
2. Query by correlation key and patch work notes [5].
3. Run the adapter twice on the lb-ns-02 verdict.
4. Show the flow assigning and notifying.

Hands-on lab. Run the pipeline twice against a failing HA pair and prove it creates one incident, then updates it with new work notes.

1. Prepare the PDI user and roles [6][43].
2. Write the query, create and patch functions [5].
3. Write the decision logic.
4. Add dry-run mode.
5. Build the flow [36].
6. Break sync on the lab NetScaler pair.
7. Run twice and check the incident.
8. Resolve and run again to test reopen.

Gennoor Tech TRAIN. INNOVATE. BUILD. For more info: https://gennoor.com

Lab success criteria.

- One incident after two runs.
- Second run added work notes.
- Dry run sends nothing.
- Reopen path routed for approval.

Deliverable. ServiceNow ticket adapter with dedup logic.

Common pitfalls.

- Static correlation values that reopen unrelated closed incidents.
- Creating before querying.
- Priority set both in code and in flow.
- Retrying POST after a timeout without checking.

Knowledge check.

1. What does POST to the Table API return on success? Answer: 201 with a Location header for the new record [5].
2. When does the adapter create a new ticket? Answer: Only when no open or recently resolved ticket matches the correlation key.
3. Why use a Due Date in Ask for Approval? Answer: To stop the flow waiting forever [36].
4. What does dry run do? Answer: Prints the payload and decision without calling ServiceNow.

Slides and assets to prepare.

- Decision table: update, reopen, create.
- Incident payload mapping.
- Flow export for the PDI.
- Postman or curl collection.

Sources for this module. 5, 6, 36, 43

Module 8: Guardrails, approvals and audit for health monitoring

Goal. Learners add approval gates, an audit record and flood protection so nothing consequential happens without a person and every run is explainable.

Outline topics.

- Read-only by default: the MCP server exposes no write tools to devices
- Approval gates for consequential actions: ticket priority P1 and P2, reopening resolved tickets, paging on-call
- Audit record for every run: readings, prompt, verdict, ticket action, approver and timestamps in SQLite
- Rate limits, timeouts and circuit breakers so a flapping device cannot flood ServiceNow

Genroor Tech TRAIN · INNOVATE · BUILD · For more info - https://genroor.com

Learning objectives.

- Learners can show the MCP server has no write tools for devices [30].
- Learners can gate P1/P2, reopen and paging behind approval.
- Learners can write a complete audit record per run in SQLite.
- Learners can add rate limits, timeouts and circuit breakers.

Read-only by default

- Only get_* tools; tools/list proves it [30].
- Agent denied Bash and write tools; PreToolUse hook blocks any mcp__devices__set* names [33][34].
- Device accounts are read-only roles.
- The MCP spec expects confirmation for sensitive operations and human ability to deny [30].

Approval gates

- Consequential: priority 1 or 2, reopening, paging on-call.
- The run pauses with status PENDING APPROVAL and a request to the NOC lead.
- Approval via ServiceNow Ask for Approval or a CLI approve command recorded in audit [36].
- Timeout leaves the ticket at priority 3 with a note.

Audit record

- Tables: runs, readings, prompts, verdicts, ticket_actions, approvals.
- Each row has run id, timestamps and hashes.
- Never store secrets; store source paths.
- Clients should log tool usage for audit [30].

Flood protection

- Per-device rate limit on ticket actions.
- Circuit breaker opens after N flaps in M minutes and writes one summary note.
- Timeouts on every device and ServiceNow call; servers must rate limit tool calls [30].
- Back off on ServiceNow errors.

Instructor demo.

1. Show tools/list output.
2. Trigger a P1 verdict and show the pause.
3. Approve and show the audit rows.
4. Flap a VIP and show the circuit breaker.

Hands-on lab. Add the approval gate and audit log, then trigger a P1 verdict and watch the run pause until a person approves.

1. Add the SQLite schema.

Genmoor Tech
For more info - https://genmoor.com/

2. Write audit calls in every stage.
3. Implement the approval gate.
4. Add the hook and deny rules [33][34].
5. Trigger a P1 bundle.
6. Approve and resume.
7. Flap a VIP to test the breaker.
8. Export the run report.

Lab success criteria.

- P1 waits for approval.
- Audit has every stage for the run.
- Breaker limits ticket actions.
- No write tools exposed.

Deliverable. Approval and audit layer.

Common pitfalls.

- Approvals stored only in chat.
- Secrets in audit rows.
- Breaker that hides a real outage without a summary.
- Gate bypass in dry-run code paths.

Knowledge check.

1. Which actions need approval? Answer: P1/P2 priority, reopening resolved tickets and paging on-call.
2. How do you prove the MCP server is read-only? Answer: tools/list shows only get_* tools [30].
3. What does the circuit breaker do? Answer: Stops repeated ticket actions for a flapping device and writes one summary.
4. Which hook event blocks a tool call? Answer: PreToolUse, with exit code 2 or permissionDecision deny [33].

Slides and assets to prepare.

- Gate flow diagram.
- SQLite schema.
- Hook script.
- Run report template.

Sources for this module. 30, 33, 34, 36

Day 3

Gennoor Tech AN INNOVATE BUILD For more info - https://gennoor.com

Module 9: Scheduling and running the workflow with GitHub Actions

Goal. Learners ship the pipeline through reviewed pull requests and run it on a schedule from a self-hosted runner, without false tickets during maintenance.

Outline topics.

- Repository layout, pull requests for every collector or rule change and required reviews
- GitHub Actions on a schedule and on demand, with secrets and a self-hosted runner that can reach the devices
- Run summaries as job artifacts and a daily health digest posted back to ServiceNow
- Handling device maintenance windows and planned downtime without false tickets

Learning objectives.

- Learners can protect main with reviews and checks [38].
- Learners can schedule and dispatch the workflow [7].
- Learners can run on a self-hosted runner that reaches the devices [37].
- Learners can suppress alerts during declared maintenance windows.

Repository and reviews

- collectors/, rules/, ticketing/, agent/, tests/, .github/workflows/.
- Every collector or rule change through a PR with required review and checks [38].
- Dismiss stale approvals and include administrators [38].
- CODEOWNERS for rules files.

Schedule and dispatch

- on.schedule with cron, UTC by default, optional timezone; minimum every 5 minutes [7].
- Avoid :00, which can be delayed under load [7].
- Runs only from the default branch [7].
- workflow_dispatch for on-demand runs.

Self-hosted runner and secrets

- Runner in the NOC network; outbound HTTPS 443 to GitHub only [37].
- runs-on labels route the job to that runner [37].
- Jobs not picked up fail after 24 hours queued [37].
- Agent runs headless with allowed_tools and dontAsk [34].

Maintenance windows

- Pull scheduled changes for each device [35].
- Readings during a window are recorded but ticket actions are suppressed.
- Suppression ends at window end; a still-unhealthy device then raises a ticket.

Gennoor Tech TRAINING NOTE BUILD For more info https://gennoor.com

- Daily digest posted to ServiceNow as a work note on a summary record.

Instructor demo.

1. Open a PR changing a rule and show required checks.
2. Show the workflow YAML with schedule and dispatch [7].
3. Show the runner online and a run picking it up [37].
4. Declare a window for lb-alt-01 and show suppression.

Hands-on lab. Ship the pipeline through a pull request, run it from Actions on a schedule and suppress alerts during a declared maintenance window.

1. Set branch protection [38].
2. Register a self-hosted runner [37].
3. Write the workflow with schedule and dispatch [7].
4. Add repository secrets.
5. Merge via PR.
6. Declare a change window in the PDI [35][43].
7. Break a VIP on the device in the window.
8. Confirm no ticket, then end the window and confirm the ticket.

Lab success criteria.

- Scheduled runs succeed on the runner.
- No unreviewed change reaches main.
- No ticket during the window.
- Ticket after the window if still unhealthy.

Deliverable. Scheduled, reviewed and versioned monitoring pipeline.

Common pitfalls.

- Cron at minute 0 [7].
- Workflow on a non-default branch never runs on schedule [7].
- Runner with inbound firewall holes it does not need [37].
- Window times without time zone.

Knowledge check.

1. What is the shortest schedule interval? Answer: Every 5 minutes [7].
2. What network access does a self-hosted runner need? Answer: Outbound HTTPS on 443 to GitHub [37].
3. Which branch do scheduled workflows use? Answer: The default branch [7].
4. What happens to readings during a maintenance window? Answer: They are recorded but ticket actions are suppressed.

Gennoor Tech
INNOVATE. BUILD.
For more info: https://gennoor.com

Slides and assets to prepare.

- Workflow YAML.
- Runner setup guide.
- Maintenance window logic diagram.
- Branch protection checklist.

Sources for this module. 7, 34, 35, 37, 38, 43

Module 10: Testing, dashboards and operations

Goal. Learners finish test suites, export readings to Prometheus and Grafana, write the operator runbook and measure outcomes.

Outline topics.

- pytest suites for collectors, severity rules and ticket decision logic with recorded API fixtures
- Exporting health readings to Prometheus and a Grafana fleet dashboard beside the ticket flow
- Operator runbook: what to do when a collector fails, when a verdict is wrong, when ServiceNow is down
- Measuring the outcome: time to ticket, duplicate rate, false positives

Learning objectives.

- Learners can run pytest suites for collectors, rules and ticket logic with fixtures.
- Learners can expose readings to Prometheus correctly for a batch job [39].
- Learners can build a Grafana fleet dashboard [40].
- Learners can write runbook entries and measure time to ticket, duplicates and false positives.

Test suites

- Collectors: fixture per endpoint, including errors.
- Rules: table-driven tests per severity rule.
- Ticketing: decision table tests with a fake ServiceNow.
- Agent: the twenty-bundle suite from day 2, run nightly.

Prometheus export

- Health runs are batch jobs; the Pushgateway is only for service-level batch outcomes [39].
- Per-device readings fit the Node Exporter textfile collector on the runner host instead [39].
- Pushed series persist until deleted, so stale devices need cleanup [39].
- Metrics: device_health_signal{device,signal,value}, run_duration_seconds, tickets_actions_total.

Grafana dashboard

- Add the Prometheus data source from Connections; local URL http://localhost:9090; Organization administrator role [40].

Gennoor Tech TRAINING INNOVATE BUILD For more info - https://gennoor.com

- Panels: fleet status grid, unknown count, sync failures, time to ticket.
- Link panels to ServiceNow incidents by correlation key.
- Dashboard JSON stored in the repository.

Runbook and measures

- Collector fails: check credentials, reachability, API change; readings show unknown.
- Verdict wrong: record case, add fixture, fix rule or prompt.
- ServiceNow down: queue actions, retry, alert by another channel.
- Measures: time to ticket, duplicate rate, false positive rate.

Instructor demo.

1. Run the full pytest suite.
2. Show the textfile collector output and Prometheus scrape [39].
3. Build two Grafana panels [40].
4. Break a collector and show dashboard, audit and runbook together.

Hands-on lab. Break a collector on purpose, watch the dashboard and the audit log explain it, and write the runbook entry.

1. Complete test suites.
2. Add metrics export [39].
3. Set up Prometheus and Grafana [40].
4. Build the dashboard.
5. Break the NetScaler collector credentials.
6. Watch unknown rise and audit errors.
7. Write the runbook entry.
8. Compute the three measures from audit data.

Lab success criteria.

- Tests pass in CI.
- Dashboard shows the broken collector as unknown.
- Runbook entry is actionable.
- Measures computed from data.

Deliverable. Test suite, dashboard and operator runbook.

Common pitfalls.

- Pushgateway for per-device metrics [39].
- Dashboards that hide unknown.
- Tests that need live devices.
- Measures computed by hand.

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

Knowledge check.

1. When is the Pushgateway appropriate? Answer: For capturing the outcome of a service-level batch job [39].
2. What role do you need to add a Grafana data source? Answer: Organization administrator [40].
3. Name the three outcome measures. Answer: Time to ticket, duplicate rate and false positive rate.
4. What happens to pushed series for a removed device? Answer: They stay until deleted through the Pushgateway API [39].

Slides and assets to prepare.

- Grafana dashboard JSON.
- Metrics naming guide.
- Runbook template.
- Measures query sheet.

Sources for this module. 39, 40

Module 11: Capstone: end-to-end device health monitoring for a mixed fleet

Goal. Teams run the full platform on the lab fleet, compare dry run and live run, and hand over with a limitations statement.

Outline topics.

- Fleet in, verdicts out, with the right ServiceNow ticket created or updated and nothing consequential without approval
- Scheduled GitHub Actions run against the full lab fleet with the approval gate and audit log exercised end to end
- Dry-run and live run compared, with duplicate-ticket and false-positive checks against the acceptance criteria
- Limitations statement, operator handover and team demo

Learning objectives.

- Learners can run the scheduled pipeline end to end with gates and audit.
- Learners can compare dry-run and live results against acceptance criteria.
- Learners can hand over with a runbook and limitations statement.

Acceptance criteria

- All five platforms read; unknowns explained.
- Correct ticket actions with zero duplicates.
- No consequential action without approval.
- Audit complete for every run.

Genmoor Tech
THINK. INNOVATE. BUILD.
For more info - https://genmoor.com

Dry run versus live

- Dry run first, compare decisions.
- Live run, compare ServiceNow state.
- Differences explained.
- False positives logged.

Handover

- Runbook, dashboard, repository, schedule.
- Limitations: platforms, signals, APIs not covered.
- Owner and review cadence.
- Demo to the room.

Instructor demo.

1. Inject three faults into the fleet.
2. Show the acceptance checklist.
3. Show a model handover pack.
4. Explain scoring.

Hands-on lab. Capstone build and demo on the full lab fleet.

1. Reset fleet and PDI.
2. Run dry run and record.
3. Run live on schedule.
4. Handle approvals.
5. Check duplicates and false positives.
6. Write limitations.
7. Demo.

Lab success criteria.

- Acceptance criteria met.
- Zero duplicates.
- Audit complete.
- Clear limitations.

Deliverable. End-to-end monitoring platform and runbook.

Common pitfalls.

- Skipping the dry run.
- Hiding unknowns.
- Vague limitations.

Knowledge check.

Gennoor Tech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com

1. What proves zero duplicates? Answer: Audit ticket actions and ServiceNow records match one incident per correlation key.
2. Why compare dry run and live? Answer: To catch differences between intended and actual ticket actions.
3. Name one limitation to state. Answer: For example a platform whose sync status cannot be read.

Slides and assets to prepare.

- Capstone brief and rubric.
- Acceptance checklist.
- Handover pack template.

Sources for this module. 5, 7, 30, 32

6. Capstone plan

A scheduled platform that collects VIP, uptime, HA, sync, version and last-change evidence from F5, NetScaler, Radware, other load balancers (Anaya LB) and Cisco ACI, lets Claude produce a cited health verdict, and creates or updates the correct ServiceNow ticket with approvals and a full audit trail.

Teams run the scheduled platform against the full lab fleet with injected faults, compare dry run and live, and hand over.

Milestones

1. All collectors green or explained
2. Triage suite passing
3. Ticket adapter idempotent
4. Gates and audit working
5. Scheduled run on the runner
6. Dashboard and runbook
7. Demo and handover

Evaluation notes

- Zero duplicate tickets and no ungated P1/P2.
- Unknowns are explicit.
- Verdicts cite readings.
- Runbook and limitations are specific.

Assessment weights

GenNoor Tech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

7. Promotional video: research behind the script

Opening scenario. The scenario of someone logging in to every device each morning is a teaching scenario drawn from the client's use case, not a measured statistic. It is realistic because sync and HA state need checking on each member [14][2], and version mismatches silently disable NetScaler HA sync [21].

GermoorTech TRAIN · INNOVATE · BUILD For more info - https://germoor.com

Claim or on-screen text	Basis
'Six signals. One verdict.'	The six signals come from the client input; the verdict is the course's HealthVerdict output [32].
'11 build steps'	Matches the 11 modules in content.py.
lb-f5-01/02 version 17.1.1, HA active/standby, sync IN-SYNC	Failover states and In Sync are real F5 values [13][14]; 17.1.1 is a lab fixture value not checked against F5 release notes.
lb-ns-01 14.1 primary sync SUCCESS; lb-ns-02 13.1 secondary sync FAILED	Mismatched versions disable HA config sync [21]. The NITRO hasync field is an enable setting; failure comes from hasyncfailurereason and haerrsyncfailure [18][2]. 'SUCCESS/FAILED' mirror the CLI Sync State shown by show ha node [19].
'vips 22/23 up'	Derived from lbvserver states [1].
apic-01 version 6.0(3), fabric health 97, nodes 12/12	fabricHealthTotal and fabricNode are the sources [26]; values are illustrative.
lb-alt-01 version 33.0, VRRP master, sync UNKNOWN	Course design: missing data shows unknown. AlteonOS 31.x-35.x lines exist [9][24]; values illustrative.
'CHG0031877 NetScaler 14.1 upgrade closed 2d ago, primary only'	Teaching scenario. NetScaler documents upgrading the secondary first, so a primary-only upgrade is a deviation the course teaches learners to flag [21].
'correlation_id=lb-ns-02:ha-sync found INC0048112'	Lookup uses Table API queries [5]. We did not find official ServiceNow documentation of the incident correlation_id field; instructors should confirm it in the PDI dictionary. Numbers are fictional.
'priority: 2 → PENDING APPROVAL (noc-lead)'	Course design: P1/P2 wait for a person; MCP guidance calls for human confirmation of sensitive operations [30].
'schedule: every 30 min, runner: noc-runner-01'	GitHub schedules allow intervals down to 5 minutes and self-hosted runners [7][37].
'maintenance window … alerts suppressed'	Course design using scheduled change records [35].
'tests: 64/64, time to ticket 4m 12s, duplicates 0'	Illustrative run output.
'5 device platforms covered'	F5, NetScaler, Radware, other load balancers (Anaya LB, via the generic adapter) and Cisco ACI. Anaya LB itself could not be found in public documentation.
'0 duplicate tickets'	Design target measured in the capstone through the correlation key logic [5].
'Flow Designer handles assignment and notification'	Flow Designer actions such as Ask for Approval are documented by ServiceNow [36].

'Export readings to Prometheus and a Grafana fleet dashboard'	Supported by Prometheus and Grafana docs [39][40].

References

[1] lbvserver (Statistics for Load Balancing Virtual Server resource). NetScaler (Cloud Software Group) developer docs. https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/statistics/lb/lbvserver
Supports: Module: NetScaler virtual server health via NITRO stat API. GET /nitro/v1/stat/lbvserver (or /lbvserver/<name>) returns per-vserver state values such as UP(7), DOWN(1), OFS(4), TROFS(5), plus vslbhealth (percentage of bound services that are UP), actsvcs and totalrequests.

[2] hanode (Statistics for HA node resource). NetScaler (Cloud Software Group) developer docs. https://developer-docs.netscaler.com/en-us/adc-nitro-api/13/statistics/ha/hanode
Supports: Module: NetScaler HA pair and config-sync status checks. GET /nitro/v1/stat/hanode exposes hacurmasterstate (PRIMARY/SECONDARY/CLAIMING/FORCE CHANGE), hacurstate (UP, DISABLED, PARTIALFAIL, COMPLETEFAIL, etc.) and haerrsyncfailure, a counter of failed primary/secondary config syncs since the last transition, commonly caused by an RPC password mismatch. Requests authenticate with a NITRO_AUTH_TOKEN cookie or X-NITRO-USER/X-NITRO-PASS headers; hacurstatus NO means the other HA statistics are invalid.

[3] iControlREST Home. F5 CloudDocs. https://clouddocs.f5.com/api/icontrol-rest/
Supports: Module: F5 BIG-IP polling via iControl REST. iControl REST is the REST successor to the SOAP iControl framework; anything doable from the BIG-IP command line can be done programmatically, with LTM objects such as pools reached under /mgmt/tm/ltm/ (example: POST to /mgmt/tm/ltm/pool with basic-auth credentials).

[4] APIRef_tm_ltm_pool_members. F5 CloudDocs. https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_ltm_pool_members.html
Supports: Module: interpreting F5 pool member state/session fields. Pool member objects expose a state field (user-down forces the member offline overriding monitors; user-up returns it to its monitor-derived state), a session field (default user-enabled, controlling new sessions) and a monitor field listing assigned health monitors and their status.

[5] Table API (Table - POST /now/table/{tableName}). ServiceNow product documentation. https://www.servicenow.com/docs/r/api-reference/rest-apis/c_TableAPI.html
Supports: Module: ServiceNow incident creation via Table API. POST /now/table/{tableName} inserts exactly one record (bulk insert not supported), accepts JSON or XML bodies via Content-Type, returns 201 with a Location header for the new record, and the calling user must hold roles sufficient to access the target table; an example posts short_description, assignment_group, urgency and impact to incident. GET /now/table/{tableName} supports sysparm_query encoded queries (^ for AND, ^OR, ORDERBY/ORDERBYDESC), sysparm_fields, sysparm_limit (default 10000) and sysparm_display_value, and PATCH /now/table/{tableName}/{sys_id} updates the named fields.

[6] Create an incident record. ServiceNow product documentation. https://www.servicenow.com/docs/r/api-reference/rest-api-explorer/t_GetStartedCreateInt.html
Supports: Module: hands-on lab creating an incident from a health-check failure. The walkthrough POSTs to https://instance.service-now.com/api/now/v1/table/incident with a body like {"short_description":"...","comments":"..."}; it requires the admin, web_service_admin or rest_api_explorer role and the response includes a Location header pointing at the new incident.

[7] Events that trigger workflows. GitHub Docs. https://docs.github.com/en/actions/writing-workflows/choosing-when-your-workflow-runs/events-that-trigger-workflows#schedule
Supports: Module: scheduling health checks with GitHub Actions cron. The schedule event uses five-field POSIX cron in UTC (a time zone can be set), minimum interval is every 5 minutes, runs only from the default branch, can be delayed at the top of the hour under load, and scheduled workflows in public repos are auto-disabled after 60 days of inactivity.

[8] Agent SDK overview. Anthropic (Claude Code docs). https://code.claude.com/docs/en/agent-sdk/overview
Supports: Module: orchestrating device checks and ticketing with a Claude agent. The Claude Agent SDK, available in Python and TypeScript, runs the same agent loop as Claude Code with built-in tools (file read/write, run commands, web search), hooks, subagents, permissions, sessions and MCP support for connecting external tools and data sources.

[9] Related Documentation (AlteonOS 31.0.11.0 Release Notes). Radware.
https://portals.radware.com/releasenotes/Alteon_Release_Notes_31.0.11.0/Alteon_Release_Notes/AlteonOS-31_0_11_0-RN.1.51.html
Supports: Module: Radware Alteon monitoring and REST API reference pointer. Radware's official Alteon documentation set includes a dedicated 'Alteon REST API User Guide' alongside the Command Reference, CLI Application Guide and Troubleshooting Guide, which are the primary references for programmatic status and uptime checks on Alteon ADCs.

[10] F5 iControl REST API reference: tm/cm/sync-status. F5 CloudDocs. https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_cm_sync-status.html
Supports: Collecting health from F5 and NetScaler. Config sync status is read from the collection /mgmt/tm/cm/sync-status (OPTIONS, GET) or a resource under it; the endpoint is read-only.

[11] F5 iControl REST API reference: tm/cm/device. F5 CloudDocs. https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_cm_device.html
Supports: Collecting health from F5 and NetScaler. /mgmt/tm/cm/device returns each device-group member with failoverState (default unknown), haCapacity, version, build, selfDevice, managementIp and timeZone.

[12] iControl REST Cookbook - Virtual Server (ltm virtual). F5 DevCentral.
https://community.f5.com/kb/technicalarticles/icontrol-rest-cookbook---virtual-server-ltm-virtual/287546
Supports: Collecting health from F5 and NetScaler. GET /mgmt/tm/ltm/virtual/<vs>/stats returns counters under entries/nestedStats including status.availabilityState (for example available), status.enabledState and status.statusReason; the tmsh equivalent is show ltm virtual.

[13] tmsh reference: sys failover. F5 CloudDocs. https://clouddocs.f5.com/cli/tmsh-reference/v16/modules/sys/sys_failover.html
Supports: Collecting health from F5 and NetScaler; What device health means across the fleet. tmsh show sys failover displays the failover state (active, standby, offline, forced_offline) and how long the unit has been in that state.

[14] Managing Configuration Synchronization (BIG-IP Device Service Clustering 14.1). F5 TechDocs.
https://techdocs.f5.com/kb/en-us/products/big-ip_ltm/manuals/product/big-ip-device-service-clustering-administration-14-1-0/05.html
Supports: What device health means across the fleet; Claude as the triage analyst. Sync statuses are In Sync, Changes Pending, Awaiting Initial Sync, Not All Devices Synced and Disconnected (iQuery channel down); sync failures are investigated in /var/log/ltm; all device group members must run the same BIG-IP software version; automatic sync still needs the config saved (save sys config or save-on-auto-sync).

[15] Auditing User Access (BIG-IP 17.0). F5 TechDocs. https://techdocs.f5.com/en-us/bigip-17-0-0/big-ip-systems-user-account-administration/auditing-user-access.html
Supports: Last change performed: correlating config history and change records. Configuration change audit messages go to /var/log/audit and login/logout messages to /var/log/secure; MCP audit levels are Disable (default), Enable, Verbose and Debug, and the tmsh audit level Enable (default) logs every tmsh command.

[16] Authentication (F5 BIG-IP Declarative Onboarding docs). F5 CloudDocs.
https://clouddocs.f5.com/products/extensions/f5-declarative-onboarding/latest/authentication.html
Supports: Collecting health from F5 and NetScaler; Setting up Claude Code, the Agent SDK and MCP for this workflow. Token auth is a POST to /mgmt/shared/authn/login with username, password and loginProviderName tmos; the token goes in the X-F5-Auth-Token header, lasts 20 minutes by default and can be extended with a PATCH to /mgmt/shared/authz/tokens/<token> up to 36000 seconds.

[17] nsversion (NITRO configuration). NetScaler developer docs. https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/ns/nsversion

Supports: Collecting health from F5 and NetScaler. GET /nitro/v1/config/nsversion returns version (read-only), mode (kernel mode KMPE/VMPE) and installedversion.

[18] hanode (NITRO configuration). NetScaler developer docs. https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/ha/hanode
Supports: Collecting health from F5 and NetScaler. The hanode configuration resource exposes state (Primary, Secondary, UNKNOWN), hastatus (ENABLED, STAYSECONDARY, DISABLED, STAYPRIMARY), hasync as an ENABLED/DISABLED setting, the read-only hasyncfailurereason, masterstatetime and routemonitorstate (UP, DOWN).

[19] ha node (NetScaler command reference). NetScaler developer docs. https://developer-docs.netscaler.com/en-us/adc-command-reference-int/13/ha/ha-node
Supports: Collecting health from F5 and NetScaler; What device health means across the fleet. show HA node output includes name, IPAddress, haStatus, state (HA master state), haSync (HA Sync State), haProp, haSyncFailureReason, masterStateTime and routeMonitorState; master states are PRIMARY, SECONDARY and CLAIMING.

[20] system (NITRO statistics). NetScaler developer docs. https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/statistics/system/system
Supports: Collecting health from F5 and NetScaler. GET /nitro/v1/stat/system returns starttime, starttimelocal and timesincestart (seconds since the appliance started) plus CPU and memory percentages such as cpuusagepcnt and memusagepcnt.

[21] Upgrade a high availability pair (NetScaler). NetScaler product documentation. https://docs.netscaler.com/en-us/citrix-adc/current-release/upgrade-downgrade-citrix-adc-appliance/upgrade-downgrade-ha-pair
Supports: Claude as the triage analyst; Last change performed. When the two HA nodes run different software versions, HA config synchronization, HA command propagation and connection mirroring are disabled (for builds of one release only when internal HA versions differ); upgrade the secondary node first and then the primary.

[22] Audit logging (NetScaler). NetScaler product documentation. https://docs.netscaler.com/en-us/citrix-adc/current-release/system/audit-logging
Supports: Last change performed: correlating config history and change records. Logs are stored locally in ns.log under /var/log/ by default with all levels except DEBUG enabled; NetScaler can also send audit messages up to 16 KB to external SYSLOG (UDP) or NSLOG (TCP) servers.

[23] Radware Alteon Load Balancer extension. Dynatrace documentation. https://docs.dynatrace.com/docs/observe/infrastructure-observability/extensions/radware-alteon-load-balancer
Supports: Collecting health from Radware, other load balancers (Anaya LB) and Cisco ACI. SNMP v2c/v3 monitoring of Alteon uses IF-MIB, ALTEON-CHEETAH-LAYER4-MIB, ALTEON-CHEETAH-NETWORK-MIB, ALTEON-CHEETAH-SWITCH-MIB and ALTEON-ROOT-MIB, with metrics for real server status per virtual service, HA service group state, HA mode and sysUpTime.

[24] Radware Alteon VA on AWS Marketplace. Radware via AWS Marketplace. https://aws.amazon.com/marketplace/pp/prodview-wknsymopbqpay
Supports: Lab environment. Radware sells Alteon VA as an AMI (listed version AlteonOS-U22-35.0.0.0-1374) with hourly usage pricing and a 30-day free trial; the listing names vDirect and Vision as automation tools that work with it.

[25] User access, authentication and accounting (Cisco ACI Fundamentals 5.1(x)). Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/5x/aci-fundamentals/cisco-aci-fundamentals-51x/m_aaa.html
Supports: Last change performed: correlating config history and change records. The aaaModLR object tracks the changes users make to objects and when they occurred, aaaSessionLR tracks logins and logouts; both can be queried by class or DN, a class query returns records for the whole fabric, and no export policy exists by default.

[26] Operating Cisco ACI appendix: REST examples. Cisco. https://www.cisco.com/c/ja_jp/td/docs/switches/datacenter/aci/apic/sw/1-x/Operating_ACI/guide/b_Cisco_Operating_ACI/b_Cisco_Operating_ACI_appendix_011.html

Supports: Collecting health from Radware, other load balancers (Anaya LB) and Cisco ACI. The appendix lists class queries /api/node/class/fabricHealthTotal.json, /api/node/class/topSystem.json, /api/node/class/fabricNode.json, /api/node/class/aaaModLR.json and /api/node/class/faultInst.json (older release; class names unchanged).

[27] Cisco APIC REST API Configuration Guide, Release 4.2(x) and Later - Using the REST API. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-rest-api-configuration-guide/cisco-apic-rest-api-configuration-guide-42x-and-later/m_using_the_rest_api.html
Supports: Collecting health from Radware, other load balancers (Anaya LB) and Cisco ACI. Clients POST aaaUser name/pwd to /api/aaaLogin.json, use the APIC-cookie token, refresh with aaaRefresh before the default 600-second timeout, and filter class queries with query-target-filter, rsp-subtree-include=faults,health and page/page-size.

[28] How Claude remembers your project (CLAUDE.md). Anthropic (Claude Code docs). https://code.claude.com/docs/en/memory
Supports: Setting up Claude Code, the Agent SDK and MCP for this workflow. Project CLAUDE.md lives at ./CLAUDE.md or ./.claude/CLAUDE.md, /init drafts one, @path imports pull in other files, keep it under about 200 lines, and it is context rather than enforced configuration; use a PreToolUse hook to block actions.

[29] Connect Claude Code to tools via MCP. Anthropic (Claude Code docs). https://code.claude.com/docs/en/mcp
Supports: Setting up Claude Code, the Agent SDK and MCP for this workflow. claude mcp add [options] <name> -- <command> [args] with --env and --scope local|project|user; project scope writes .mcp.json, which supports ${VAR} and ${VAR:-default} expansion so secrets stay in the environment; /mcp shows server status.

[30] Model Context Protocol specification: Tools. Model Context Protocol. https://modelcontextprotocol.io/specification/2025-06-18/server/tools
Supports: Setting up Claude Code, the Agent SDK and MCP for this workflow; Guardrails, approvals and audit for health monitoring. Tools are listed with tools/list and called with tools/call; definitions carry inputSchema, optional outputSchema and annotations that clients treat as untrusted; results can include structuredContent and isError; servers MUST validate inputs and rate limit, clients SHOULD confirm sensitive operations, apply timeouts and log tool usage for audit.

[31] MCP Python SDK. Model Context Protocol. https://github.com/modelcontextprotocol/python-sdk
Supports: Setting up Claude Code, the Agent SDK and MCP for this workflow. Install with pip install "mcp[cli]" (Python 3.11+), register tools with @mcp.tool() using type hints and docstrings for the schema, run over stdio or Streamable HTTP, and test with uv run mcp dev server.py in the MCP Inspector.

[32] Get structured output from agents. Anthropic (Claude Agent SDK docs). https://code.claude.com/docs/en/agent-sdk/structured-outputs
Supports: Claude as the triage analyst: structured health verdicts. output_format={"type": "json_schema", "schema": Model.model_json_schema()} returns validated data in structured_output on the ResultMessage; the SDK validates with JSON Schema draft-07 and retries, failing with error_max_structured_output_retries; a success result without structured_output must also be treated as a failure.

[33] Hooks reference. Anthropic (Claude Code docs). https://code.claude.com/docs/en/hooks
Supports: Guardrails, approvals and audit for health monitoring. PreToolUse hooks run before a tool call and block it with exit code 2 or permissionDecision deny; MCP tools are matched as mcp__<server>__<tool> or mcp__<server>__.*; PostToolUse runs after success and cannot block.

[34] Configure permissions (Agent SDK). Anthropic (Claude Agent SDK docs). https://code.claude.com/docs/en/agent-sdk/permissions
Supports: Setting up Claude Code, the Agent SDK and MCP for this workflow; Scheduling and running the workflow with GitHub Actions. Evaluation order is hooks, deny rules, ask rules, permission mode, allow rules, canUseTool; disallowed_tools removes tools; allowed_tools with permission_mode dontAsk gives a fixed tool surface for headless agents; allowed_tools does not constrain bypassPermissions.

[35] Change Management API. ServiceNow. https://www.servicenow.com/docs/r/api-reference/rest-apis/change-management-api.html

Supports: Last change performed: correlating config history and change records. GET /sn_chg_rest/change retrieves change requests with sysparm_query, sysparm_offset and order; GET needs change_manager, itil, sn_change_read or admin; change model states include New, Scheduled, Implement, Review and Closed.

[36] Ask for Approval action (Workflow Studio / Flow Designer). ServiceNow. https://www.servicenow.com/docs/r/build-workflows/workflow-studio/ask-approval-flow-designer.html
Supports: Creating or updating ServiceNow tickets; Guardrails, approvals and audit for health monitoring. Ask for Approval takes Record, Table, Approval Reason, Approval Field, Journal Field, Rules and a Due Date to stop endless waiting; Approval State output values are Not Yet Requested, Requested, Approved, Rejected, Cancelled, No Longer Required and Skipped; flow_designer or admin role required.

[37] Self-hosted runners reference. GitHub Docs. https://docs.github.com/en/actions/reference/runners/self-hosted-runners
Supports: Scheduling and running the workflow with GitHub Actions. Self-hosted runners connect to GitHub to receive jobs and need outbound HTTPS on port 443; a job not picked up is re-queued after 60 seconds and fails if queued for more than 24 hours; jobs are routed by runs-on labels and groups.

[38] About protected branches. GitHub Docs. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
Supports: Scheduling and running the workflow with GitHub Actions. Protected branches can require approving reviews, passing status checks, code owner review and dismissal of stale approvals, and can apply the rules to administrators.

[39] When to use the Pushgateway. Prometheus. https://prometheus.io/docs/practices/pushing/
Supports: Testing, dashboards and operations. The Pushgateway's valid use is capturing the outcome of a service-level batch job; it is a single point of failure, loses the automatic up metric and never forgets pushed series unless deleted; machine-related batch jobs should use the Node Exporter textfile collector instead.

[40] Configure the Prometheus data source (Grafana). Grafana Labs. https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/
Supports: Testing, dashboards and operations. Add the data source from Connections > Add new connection > Prometheus > Add new data source; a local server URL is http://localhost:9090; the Organization administrator role is required.

[41] NetScaler licensing (VPX Express). NetScaler product documentation. https://docs.netscaler.com/en-us/citrix-adc/current-release/licensing
Supports: Lab environment. VPX Express needs no license file, is limited to 20 Mbps throughput and 250 SSL sessions, and from 14.1 build 29.65 includes the Premium feature set.

[42] ACI Sandboxes. Cisco DevNet. https://developer.cisco.com/docs/aci/sandbox
Supports: Lab environment. DevNet offers a shared, no-reservation ACI Simulator (Always-On) for APIC API requests and a reservable ACI Simulator for up to 5 consecutive days, at zero cost.

[43] Get a development instance (ServiceNow PDI). ServiceNow. https://www.servicenow.com/docs/r/9Lur8HBNLjdzEcjznSAFzA/HTEzwTEvNIUFLxGw~Sq_3A
Supports: Lab environment. A free personal developer instance is obtained from the ServiceNow Developer Site by following the Personal Developer Instance (PDI) guide.

[44] Verify the firmware version and the upgrade status using the API. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/g-upgrading-the-software-using-the-rest-api/verifying-firmware-version-and-upgrade-status.html
Supports: Collecting health from Radware, other load balancers (Anaya LB) and Cisco ACI. Controller firmware is read from /api/node/class/firmwareCtrlrRunning, switch firmware from /api/node/class/firmwareRunning and upgrade jobs from /api/node/class/maintUpgJob.

[45] Try BIG-IP and BIG-IQ Virtual Editions. F5. https://www.f5.com/trials/big-ip-virtual-edition
Supports: Lab environment. A free 30-day trial from MyF5 > My Products & Plans > Trials includes one BIG-IP VE with LTM and other modules, limited to 1 Mbps throughput, up to 4 vCPUs and 2 GB memory per vCPU.

Gennoor Tech
TRAIN. INNOVATE. BUILD.
For more info - https://gennoor.com